

Centralized Object Storage Architecture

Proposal for Managing Images, PDFs, Documents and Other Media Across Multiple Websites

Executive Summary

The organization operates multiple websites using different technology stacks, including Laravel, React + Node.js, and Next.js. As these websites accumulate images, PDFs, documents and other uploaded media, storing files directly on individual application servers creates disk-space, backup, scalability and maintenance challenges.

This proposal recommends moving persistent media storage from application servers to centralized cloud Object Storage. Cloudflare R2 is recommended as the primary option, while Amazon S3 and DigitalOcean Spaces can be considered as alternatives.

Proposed Architecture

All websites will continue running on their existing application infrastructure. Uploaded media will be stored in centralized Object Storage and delivered to users through a CDN where appropriate.

Websites → Object Storage → CDN → Users

Laravel websites, Node.js applications, Next.js applications and future projects can all use the same S3-compatible storage architecture.

Recommended Storage Structure

A centralized bucket can be organized using website-specific prefixes/folders. This provides logical separation without requiring a separate storage system for every website.

Storage	Example Content
educationmalaysia/	universities/, courses/, blogs/, documents/
innayatmedical/	doctors/, blogs/, documents/
website-3/	images/, documents/
website-4/	images/, documents/

Application and Database Design

The actual file should not be stored in the application database. The database should store only the object key/path and relevant metadata, such as filename, MIME type, size, visibility and upload information.

Example: The database stores *educationmalaysia/universities/abc.webp*, while the actual file is stored in Object Storage.

Technology Compatibility

Technology	Integration
------------	-------------

Laravel / PHP	S3-compatible filesystem / SDK
Node.js	S3-compatible SDK
Next.js	S3-compatible SDK and CDN URLs
React	CDN URLs or signed URLs for protected files

Key Benefits

- Reduces dependency on application-server disk space.
- Allows all websites to share a centralized media infrastructure.
- Simplifies server migration and replacement.
- Separates application data from uploaded media.
- Improves scalability as media volume grows.
- Enables CDN-based delivery for faster image and file access.
- Provides a cleaner backup and disaster-recovery strategy.
- Supports multiple application technologies through the S3-compatible API.

Public vs. Private Files

Public assets: Images, logos and other publicly accessible content can be delivered through a CDN/custom domain.

Private files: Sensitive PDFs and documents can remain private and be accessed through authenticated application requests and temporary signed URLs.

Recommended Approach

Primary recommendation: Cloudflare R2. It is well suited to a multi-website architecture because it provides S3-compatible object storage and integrates naturally with Cloudflare's ecosystem. Amazon S3 is the strongest alternative where the organization already has significant AWS infrastructure. DigitalOcean Spaces is another straightforward option.

Implementation Roadmap

1. Create and configure the Object Storage account and bucket.
2. Define website-specific prefixes and access policies.
3. Configure CDN/custom domains where required.
4. Integrate Laravel applications with the storage backend.
5. Integrate Node.js and Next.js applications using the S3-compatible API.
6. Update upload logic so persistent files are stored remotely rather than on application servers.
7. Update file URLs and download logic.
8. Migrate existing media from current servers in a controlled manner.
9. Verify permissions, backups, URLs and application functionality.
10. Remove or reduce old local media storage after successful verification.

Management Review / Decision Points

Management approval is requested to evaluate centralized Object Storage as the standard media-storage architecture for current and future websites. The technical team can then prepare a

detailed cost estimate, security policy, migration plan and proof-of-concept before production rollout.